Energy-Efficient Large-Scale Vector Similarity Search in NAND-Flash via Hybrid Matching

Chih-Yu Hu[1§], Chi-Tse Huang[2§], Hao-Wei Chiang[2], Hsiang-Yun Cheng[3],
Po-Hao Tseng[4], Ming-Hsiu Lee[4], An-Yeu (Andy) Wu[2]
[1]*Department of Mechanical Engineering, National Taiwan University, Taipei, Taiwan*
[2]*Graduate Institute of Electronics Engineering, National Taiwan University, Taipei, Taiwan*
[3]*Research Center for Information Technology Innovation, Academia Sinica, Taipei, Taiwan*
[4]*Macronix International Co. Ltd, 16 Li-Hsin Road, Hsinchu Science Park, Taiwan*
b09502137@ntu.edu.tw, {rickhuang, jackson}@access.ee.ntu.edu.tw, hycheng@citi.sinica.edu.tw,
pohaotseng@mxic.com.tw, Ericlee@mxic.com.tw, andywu@ntu.edu.tw

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132636

***Abstract*—Vector similarity search (VSS) is crucial in many AI applications, such as few-shot learning (FSL) and approximate nearest-neighbor search (ANNS), but it demands significant memory capacity and incurs substantial energy costs for data transfers during large-scale comparisons. Various in-memory search technologies have been developed to improve energy efficiency, with NAND-based multi-bit content-addressable memory (MCAM) standing out as a promising solution for its high density and large capacity. MCAM can operate in exact-search (ES) mode, supporting only perfect matches with low energy cost, or in approximate-search (AS) mode, enabling flexible VSS. However, AS mode incurs significant energy waste when comparing queries with non-target stored vectors. To address this issue, we propose Hybrid-M, a 3D NAND-based in-memory VSS architecture that integrates both modes into a single hybrid matching process, using ES mode as a filter to reduce redundant searches for AS mode. We apply three techniques to optimize this integration: range encoding for multi-level cells (MLC) to enhance filtering, search voltage shifts to mitigate the impact on AS accuracy and reduce matching currents, and a filtering-aware training method to further improve reliability and energy efficiency. Results show that Hybrid-M achieves comparable accuracy while reducing energy consumption by 67% to 83% compared to MACM-based VSS using only AS mode, across various many-class FSL and ANNS workloads.**

I. Introduction

Vector similarity search (VSS), which compares a query with stored vectors using specific similarity metrics, is a cornerstone of many applications. It has become crucial in AI, where neural networks and machine learning models encode the semantics of images, videos, and documents as high-dimensional vectors—often called embeddings or features—and use their similarities for predictions. In practice, VSS is often scaled to accommodate large datasets. Prominent examples include many-class few-shot learning (FSL) [1], [2], which classifies unseen queries from numerous classes, and approximate nearest-neighbor search (ANNS) [3], [4], extensively used in recommendation systems and retrieval-augmented generation (RAG) [5] for efficient semantic searches.

Despite its pivotal role in modern AI applications, VSS often faces performance and energy-efficiency bottlenecks due to substantial memory demands and frequent data transfers [6], [7]. For instance, production-level recommendation systems,

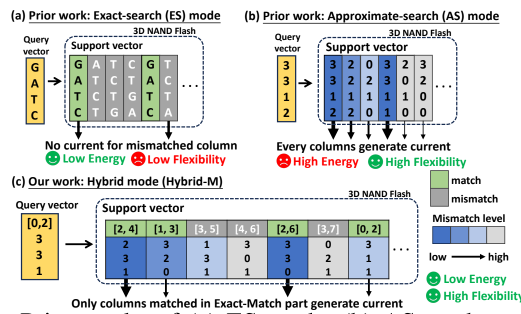

Fig. 1: Prior works of (a) ES mode, (b) AS mode, and (c) the proposed work with Hybrid-M.

such as Microsoft Bing [8] and Alibaba e-commerce platforms [9], need tens of TB memory for ANNS with billions of 100+ dimensional vectors. The sheer volume of high-dimensional data makes VSS memory- and bandwidth-intensive, requiring significant energy efficiency optimizations.

Researchers have developed in-memory search (IMS) architectures using various memory technologies to tackle this challenge [10]–[12]. Given the significant memory demands of large-scale VSS, 3D NAND-based multi-bit content-addressable memory (MCAM) emerges as a promising solution due to its high density, large storage capacity, and support for highly parallel search operations. Studies have shown that 3D NAND-based MCAM, with appropriate search voltage settings, supports two modes: exact-search (ES) [13] and approximate-search (AS) [14]. The low-energy ES mode, depicted in Fig. 1(a), identifies only vectors identical to the query, while the AS mode, shown in Fig. 1(b), flexibly identifies vectors similar to the query, with currents correlating to their distance from the query. However, the AS mode incurs substantial energy waste during comparisons between the query and non-target vectors. Our analysis reveals that nearly 98.07-99.61% of energy is wasted in many-class FSL and ANNS scenarios. This trade-off between energy efficiency and search flexibility highlights the need for optimized search flows in 3D NAND-based IMS architectures to achieve energy-efficient large-scale VSS.

To this end, we propose Hybrid-M, a 3D NAND-based IMS architecture that seamlessly integrates ES and AS modes into a single hybrid matching process, as shown in Fig. 1(c). The

[§]Equal contribution

ES mode acts as a filter to reduce redundant searches for the AS mode. Unlike other IMS designs that use separate arrays for each mode [15], Hybrid-M leverages the serial-connection structure of 3D NAND flash to concatenate both modes within the same NAND string. By applying different search voltages to NAND cells associated with each mode, Hybrid-M enables searches in a single hybrid matching operation, significantly reducing latency and energy consumption.

Integrating these two modes within the same NAND string poses several design challenges. First, allocating a portion of NAND cells to the ES mode reduces the available storage for the AS mode, limiting the vector dimensions and potentially impacting accuracy. To avoid significantly compromising AS performance, only a limited number of NAND cells can be dedicated to the ES mode. Within this constraint, the filtering dimensions must be carefully selected and efficiently encoded to maximize the filtering rate. Second, the presence of the ES mode may alter the currents in both modes, which disrupts the correlation between current levels and the distance to the query.

In this paper, we apply four techniques to address these challenges. First, we select the representative dimension with the maximum inter-class variance and program it with ES mode, enabling filtering based on the L_∞ similarity metric [16]. Second, we design a range encoding scheme for multi-level cells (MLC) to improve the filtering rate with limited cells for ES mode. Third, to mitigate the impact of the hybrid-matching mechanism on AS reliability, we lower the search voltage. This adjustment also reduces the overall current in each NAND string, thereby lowering search energy. Finally, we introduce a filtering-aware training method that incorporates the hybrid-matching mechanism with filtering into the training process. This approach further improves VSS accuracy and energy efficiency, particularly for applications like many-class FSL, where feature vectors are generated using neural networks. The key contributions of this work are as follows:

1) **Combining ES and AS modes to filter redundant current during VSS in 3D NAND flash without overhead** : Hybrid-M reduces 20% to 52% in energy consumption by applying filtering in the hybrid matching process during VSS, as compared to using the AS mode alone.
2) **Enhancing filtering capability via MLC range encoding under WLs constraints in 3D NAND flash** : Filtering with MLC range encoding achieves 10% to 30% additional energy savings compared to SLC range encoding.
3) **Mitigating accuracy loss in hybrid mode via search voltage shifting** : Shifting the search voltage to a lower level recovers accuracy by 10% to 15% and further reduces energy consumption by 10% to 20%.
4) **Filtering-aware training (FAT) incorporates VSS and filtering during training to reduce energy consumption** : Hybrid-M with FAT further achieves a 3% to 8% in energy consumption and a 0.5% to 1% improvement in accuracy compared to Hybrid-M without FAT.

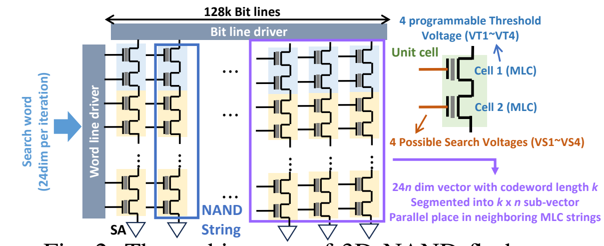

Fig. 2: The architecture of 3D NAND flash array.

II. Background and Motivation

A. Vector Similarity Search (VSS)

Vector similarity search (VSS) compares a query vector with stored vectors to identify the closest vector with similarity metrics. Common similarity measures include cosine similarity for angular comparison, as well as the L_1 [10] and L_∞ [17] norms for distance. VSS is integral to a wide range of AI applications, particularly in many-class few-shot learning (FSL) and approximate nearest-neighbor search (ANNS).

In many-class FSL, models are tasked with making classification using minimal labeled data. This is typically framed as N-way K-shot setting, where N classes with K labeled examples for each class are provided. VSS matches novel inputs (queries) against stored support vectors to infer class based on the closest matches. While cosine similarity is widely adopted in software-based implementations, its high computational complexity limits its scalability in hardware-based implementations. As a result, hardware-friendly alternatives such as the L_1 and L_∞ norms are gaining traction, offering reduced computational overhead while maintaining classification accuracy. On the other hand, ANNS [3], [4] is widely applied in scenarios requiring efficient retrieval from large, high-dimensional datasets. A critical performance metric for ANNS is the recall rate, expressed as N @ M. This metric reflects the proportion of top-N nearest-neighbors correctly identified within M retrieved vectors. High recall rates are essential for maintaining accuracy while leveraging approximation techniques to optimize computational efficiency. This paper explores VSS applications within both many-class FSL and ANNS, focusing on optimization for efficient and scalable implementations in extensive AI tasks.

B. 3D NAND Flash Architecture

Fig. 2 depicts the architecture of 3D NAND flash. Two cells are connected in series to form a word, and multiple words are concatenated within a NAND string. Each multi-level cell (MLC) word encodes two bits via complementary threshold voltages (VT). Search voltages (VS) applied to word-line gates enable parallel search across MLCs in the same row. 3D NAND flash supports two modes for in-memory search (IMS) : exact-search (ES) [13] and approximate-search (AS) [14]. The primary distinction between them lies in the voltage levels of VS. Table I lists the VT and VS combinations for MLC words, and Fig. 3(a) shows their voltage distributions.

In ES mode, VSi is carefully positioned above VTi but below the next VT (e.g., VS1 is above VT1 yet below VT2). Under mismatch, at least one VS falls below its corresponding VT, rendering the cell non-conducting and eliminating string current. As a result, ES mode is highly energy-efficient and

TABLE I: Quinary state representation for MLC.

Quinary State	Threshold Voltage		Search Voltage	
	Cell 1	Cell 2	Cell 1	Cell 2
00	VT1	VT4	VS1	VS4
01	VT2	VT3	VS2	VS3
10	VT3	VT2	VS3	VS2
11	VT4	VT1	VS4	VS1
xx	VT1	VT1	VS4	VS4

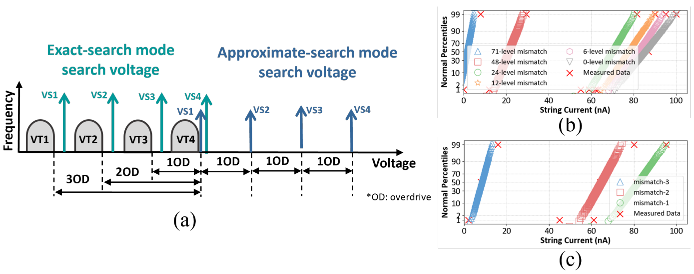

Fig. 3: (a) VS and VT settings for MLC in both ES and AS modes. Simulated current for AS mode (b) with varying string mismatch levels and (c) with identical string mismatch levels but differing maximum cell mismatches.

particularly suitable for applications like DNA sequence matching [13]. In contrast, AS mode sets all VS values above the maximum VT (e.g., VS1 is above VT4). In the presence of mismatches, the reduced voltage difference between VS and VT results in lower overdrives (ODs), decreasing the matching current. This behavior enables NAND flash to generate currents to approximate similarity in AS mode. The current is proportional to the ODs between search voltages and threshold voltages of NAND strings. Fig. 3(b) shows the current distribution of a 48-layer NAND string with mismatch levels ranging from 0 to 72 (24 words with up to 3 mismatch levels per word). As the mismatch level increases, the matching current decreases, reflecting greater distances between the query and stored vectors. Furthermore, Fig. 3(c) demonstrates that even with the same total mismatch levels, a single large mismatch in one cell (mismatch-n) can significantly reduce the overall current. To address this issue, prior work [18] proposed the multi-bit thermometer code (MTMC), which distributes mismatches more evenly across cells to improve reliability and accuracy. In this work, we also adopt MTMC in AS mode for VSS.

C. Motivational Experiments in Large-scale VSS

The high storage density of 3D NAND flash makes it ideal for IMS on large datasets. Prior work [14], [18] operates 3D NAND flash in AS mode to reduce data movement for large-scale VSS. However, 3D NAND flash incurs significant energy overhead since NAND strings always conduct regardless of mismatch levels in AS mode. Although comparisons of irrelevant data result in higher mismatch levels and smaller

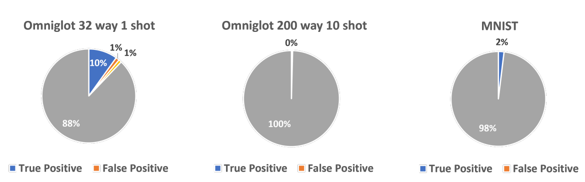

Fig. 4: Current breakdown in VSS using only AS mode.

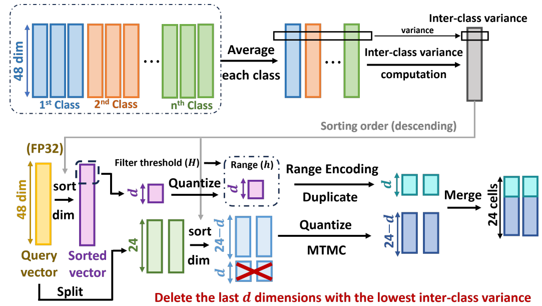

Fig. 5: Vector generation with d dimensions for filtering.

currents, their large volume leads to substantial unnecessary energy consumption. This inefficiency becomes critical in extensive comparison scenarios such as many-class FSL and ANNS, where redundant matching currents dominate energy consumption, as shown in Fig. 4.

Energy consumption in VSS can be categorized into four cases. ”True Positive” occurs when the class with the largest current correctly corresponds to the target class, representing necessary energy usage. ”False Positive” and ”False Negative” indicate classification errors: ”False Positive” occurs when the largest current corresponds to an incorrect class, leading to unnecessary energy usage; ”False Negative” happens when the correct class does not yield the largest current, resulting in misclassifications despite the energy expended. The ”True Negative” case, constituting the majority of currents, includes currents that neither correspond to the correct class nor represent the largest similarity, signifying substantial and unnecessary energy consumption.

Fig. 4 compares traditional FSL (left) and many-class FSL (middle), highlighting substantial energy inefficiency in the latter. As class count grows, a larger protion of energy is wasted on ”True Negative” matching currents, exacerbating energy challenges and emphasizing the need for scalable, energy-efficient designs. To mitigate this, prior work [15], [19] applied filtering as a separate stage to reduce the energy of VSS. In this approach, filtering is performed on TCAM using the L_∞ norm, followed by VSS in compute-in-memory (CIM) using L_1 norm computation. While effective, this approach introduces additional hardware complexity and overhead. In contrast, designing a hybrid process with 3D NAND Flash using ES mode for filtering and AS mode for VSS can eliminate dedicated filtering hardware and reduce both energy and complexity without performance loss.

III. Proposed Method

In this paper, we propose techniques used in Hybrid-M for reducing VSS energy in 3D NAND flash. In Section III.A, we introduce an ES mode filtering mechanism that uses SLC range encoding, eliminating unnecessary currents by combining ES and AS modes. In Section III.B, we extend range encoding to MLC, increasing filtering capacity without increasing codeword length. In Section III.C, we lower VS by one OD in AS mode, counteracting the accuracy loss introduced by ES mode while further reducing current magnitudes. Finally, Section III.D presents a filtering-aware training strategy to improve accuracy and enhance energy efficiency.

A. Hybrid Matching Process Combining ES and AS modes

To mitigate the high energy consumption of AS mode without incurring additional area overhead, we propose a hybrid matching scheme that leverages both ES and AS modes in 3D NAND flash. We program a portion of the WLs in ES mode to filter irrelevant vectors by selectively disabling string currents, thereby reducing energy consumption. The remaining WLs operate in AS mode, performing VSS for similarity evaluation. As illustrated in Fig. 1(c), NAND cells in AS mode produce currents proportional to mismatch levels, enabling precise VSS, while NAND cells in ES mode filter irrelevant vectors, yielding zero current for the corresponding NAND strings. This hybrid approach achieves low-power operation and ensures that only relevant strings engage in VSS.

Vector generation with d-dimensional filtering operates as shown in Fig. 5. Each NAND string contains 24 MLC words with the first d words programmed in ES mode. To minimize the number of conducting strings after filtering, the top-d dimensions with the highest inter-class variance are selected as feature dimensions for filtering. A vector is considered potentially similar only if the difference in its corresponding input falls within a predefined threshold H. To enable threshold-based filtering in NAND cells using ES mode, each selected dimension is represented as a range and encoded using range encoding [15]–[17], [20]. For AS mode, we adopt MTMC for VSS, which converts each dimension to several codewords. These codewords are mapped across multiple NAND strings. To ensure consistent filtering of the vectors, identical d-dimensional filtering dimensions are applied to all strings of the same vector. If any filter mismatch occurs, the vector is considered dissimilar, and all associated strings are deactivated. Only the remaining active strings undergo VSS in AS mode.

B. Range Encoding for MLC

Programming WLs to ES mode in a fixed-length NAND string inevitably sacrifices the capacity available for AS mode, making extensive use of ES mode undesirable. Given the limited number of WLs, improving filtering effectiveness becomes a critical issue. A naive approach is to concatenate pairs of codewords from SLC-based range encoding and map them to MLC states. However, current MLC programming schemes cannot represent all 2-bit combinations of 0, 1, and x. Prior work has demonstrated that MLC words can represent quinary states (00, 01, 10, 11, xx) [13], as shown in Table I. We extend the programmable states to include 0x, x1, and 1x for enhancing the flexibility of codeword matching, as shown in Table II. The state x0 is infeasible due to VT and VS constraints, preventing direct support for all 2-bit SLC codewords in MLC.

Building on the approach of BORE [17], which introduces constrained programming (CP) to solve constraint satisfaction in range encoding, we apply CP to generate MLC codewords represented in octal states, enabling more flexible and precise matching. Due to limited storage per string, only one MLC word can be programmed for each dimension. CP produces codewords that are prioritized based on importance, and we select the most representative one for programming. When the threshold H is large, the resulting value range becomes

TABLE II: Octal state representation for MLC.

Octal State	Threshold Voltage		Search Voltage	
	Cell 1	Cell 2	Cell 1	Cell 2
00	VT1	VT4	VS1	VS4
0x	VT1	VT3	VS2	VS4
01	VT2	VT3	VS2	VS3
x1	VT2	VT2	VS3	VS3
11	VT3	VT2	VS3	VS2
1x	VT3	VT1	VS4	VS2
10	VT4	VT1	VS4	VS1
xx	VT1	VT1	VS4	VS4

TABLE III: Mismatch level of AS mode.

w/o VS Shifting					**w/ VS Shifting**				
state	**1**	**2**	**3**	**4**	**state**	**1**	**2**	**3**	**4**
1	0	1	2	3	**1**	1	2	3	
2	1	0	1	2	**2**	2	1	2	3
3	2	1	0	1	**3**	3	2	1	2
4	3	2	1	0	**4**		3	2	1

TABLE IV: Mismatch level of ES mode.

Octal state	**00**	**0x**	**01**	**x1**	**11**	**1x**	**10**	**xx**
00	3	3						3
0x	3	2	3	3				2
01		3	3	3				2
x1		3	3	2	3	3		1
11				3	3	3		2
1x				3	3	2	3	2
10						3	3	3
xx	3	2	2	1	2	2	3	0

wider, increasing the number of "don't care" (xx) bits and reducing filtering capability. To address this, we partition H into $h_s = \left\lceil \frac{H}{2} \right\rceil$ for stored vectors and $h_q = \left\lfloor \frac{H}{2} \right\rfloor$ for query vectors. This transformation converts the original point-in-range comparison into a range-to-range matching scheme, which maintains the effective matching interval for filtering while reducing the required codeword length.

C. Shifting of Search Voltage in AS mode

In the hybrid matching process, active NAND strings are affected by mismatch levels from both ES and AS modes. The mismatch level is defined as the minimum OD between VT and VS within an MLC word, where a smaller OD corresponds to a higher mismatch level, with 0–3 OD mapped to levels 3–0, respectively. Table III and Table IV show mismatch levels introduced by AS and ES modes, respectively. As shown in Fig. 3(b)(c), larger maximum mismatch level—particularly level 3—significantly reduces matching current. Although NAND cells in AS mode produce currents proportional to similarity, the mismatch levels 2 and 3 from NAND cells in ES mode disrupt this correlation, degrading VSS accuracy. The effect of mismatch level 3 is especially severe, causing disproportionate current drops. Furthermore, the "don't care" (xx) codeword in ES mode spans multiple VT intervals, resulting in mismatch levels ranging from 0 to 3 without meaningful similarity correlation. This increases current inconsistency and further degrades matching accuracy.

To address these issues, we first remove the "don't care" from octal states and generate the codeword of range encoding using CP accordingly. We then shift VS in AS mode by one OD to mitigate the impact of high mismatch levels from NAND cells in ES mode, as shown in Fig. 6(a). This raises the baseline mismatch level, reducing the relative influence of ES

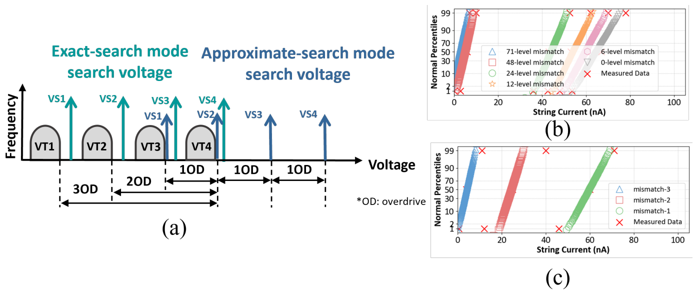

Fig. 6: (a) VS shifting in AS mode with NAND flash. AS mode simulated current during VS shifting for (b) varying string mismatch levels and (c) identical string mismatch levels with differing maximum cell mismatches, both based on [14].

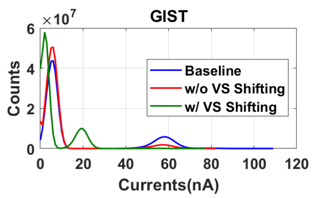

Fig. 7: Current distribution under different AS mode settings.

mismatches on matching current. As demonstrated in Fig. 7, without VS shifting (red), the right-side current peak from AS-only matching (blue) is suppressed, increasing SA errors. In contrast, VS shifting (green) restores the peak, improves current separation, and enhances VSS accuracy. The new mismatch range in AS mode spans levels 1–3, with extreme mismatches denoted by a slash in Table III. This also reduces overall current magnitude, lowering energy consumption without sacrificing performance.

D. Filtering-aware Training (FAT)

In our Hybrid-M approach, substituting AS mode vector dimensions with ES mode introduces two challenges. First, directly omitting AS mode dimensions can degrade accuracy. Second, selecting ES mode dimensions solely by inter-class variance may not yield effective filtering. We propose FAT to address both issues, preserving accuracy through optimized dimension removal while ensuring robust ES mode filtering.

FAT is a training strategy designed for tasks where vectors are extracted by neural networks. We develop FAT based on the hardware-aware training (HAT) framework [18], which incorporates flash characteristics into training for improving robustness under device non-idealities. Specifically, current variations, quantization errors, and MTMC behavior are modeled during meta-training to consider MCAM hardware. We further introduce a third stage—hybrid search–aware training—that accounts for behavioral differences between ES and AS modes, enabling the controller to adapt to hybrid matching and improving both reliability and energy efficiency.

The hybrid search-aware training, illustrated in Fig. 8, integrates L_∞ filtering to generate dimensionally reduced filter vectors that preserve classification accuracy and enhance energy efficiency. Begin by reloading the meta-training stage model and adjusting initial weight and bias values for the filtering mechanism. First, we compute inter-class and intra-class variances from the support vectors. For each NAND string containing 24 MLC words, we define subsequent vector dimensions in multiples of 24, applying a d-dimensional filter. Consequently, in a $D = 24n$ dimensional vector, $n \cdot d$ dimensions with the smallest inter-class variance are eliminated, freeing space for ES mode weights and biases. Next, to improve generalization under distribution shifts between training and testing, we select d filtering dimensions from the top $u \cdot d$ entries with the lowest intra-class variance. The averaging factor u controls selection diversity; larger values are preferred when the training–testing gap is wider. These entries are partitioned into d groups and averaged to derive representative weights, providing a robust initialization aligned with the overall data distribution. During the hybrid search-aware training, we introduce a learnable threshold $\mathbf{T}_{ES}$ and gating function $\mathbf{G}_{ES}(\cdot)$ to simulate the L_∞ filtering effect. Given the ES mode distance $\mathbf{D}_{ES}$ and AS mode similarity $\mathbf{S}_{AS}$, the final similarity outcome $\mathbf{S}_O$ is specified by:

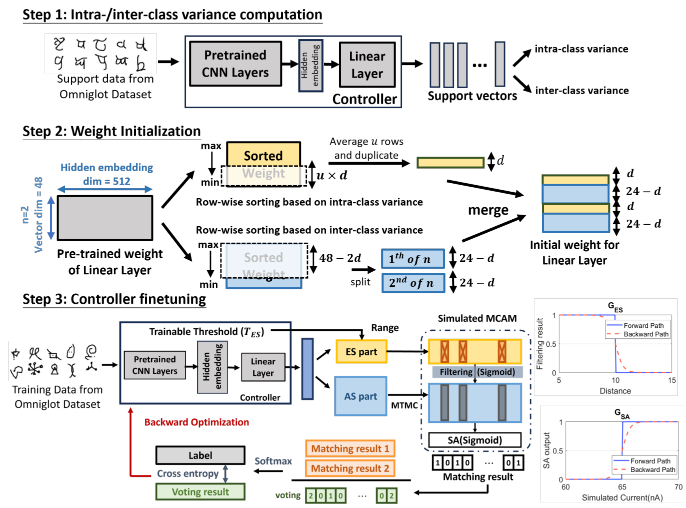

Fig. 8: Training Flow of proposed Filtering-aware Training.

$$\mathbf{S}_O = \mathbf{S}_{AS} \cdot \mathbf{G}_{ES}(\mathbf{D}_{ES} \leq \mathbf{T}_{ES}) = \begin{cases} \mathbf{S}_{AS}, & \mathbf{D}_{ES} \leq \mathbf{T}_{ES} \\ 0, & \mathbf{D}_{ES} > \mathbf{T}_{ES} \end{cases}$$

since the gating function is non-differentiable around the threshold, it leads to gradient vanishing during backpropagation. To address this, a sigmoid function is applied during training to preserve gradient flow. By leveraging FAT, the integration of ES and AS modes yields robust, hardware-aware solutions for large-scale VSS.

IV. Evaluation

A. Experimental Setup

We evaluate our design on two FSL tasks: Omniglot [22] and CUB-200-2011 (CUB) [23]. For Omniglot, we employ a Conv4 [24] model with 48 dimensions and set the average number $u = 3$ under a 200-way 10-shot setting, which requires 128k NAND strings with a codeword of length 31 for AS mode. For CUB, we adopt a ResNet12 [25] model with 480 dimensions and $u = 10$ under a 50-way 5-shot setting, which needs 125k NAND strings and a codeword of length 25 for AS mode.

We also evaluate three ANNS tasks—Fashion-MNIST [26], MNIST [27], and GIST [28]—using L_2 norm as ground truth [3], [4] to retrieve the top 100 nearest neighbors. Fashion-MNIST and MNIST have 60,000 training and 10,000 test samples with 784 dimensions, while GIST has 1,000,000 training

TABLE V: Setting and statistic of MCAM simulation.

Device	3D NAND Flash
Capacity (KB)	750
Number of WL	48
Number of BL	128k
Search time(us)	50
Supply Voltage(V)	0.5
VT1 - VT4 (V)	[2, 3, 4, 5]
AS mode VS1 - VS4(V)	[4.5, 5.5, 6.5, 7.5]

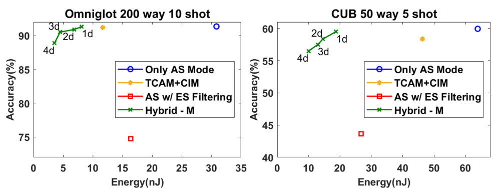

Fig. 9: Energy evaluation of many-class FSL.

and 1,000 test samples with 960 dimensions. Fashion-MNIST and MNIST employ a codeword of length 5 (requiring 9,900k NAND strings), whereas GIST uses a codeword of length 9 for AS mode (360,000k NAND strings). Since these datasets are composed of pre-trained vectors, FAT is not applied.

The estimates of energy consumption for NAND flash are based on measured current distributions [14]. The setting and statistics of MCAM simulations are listed in Table V.

B. Reduction of Energy

Fig. 9 and Fig. 10 show evaluations on many-class FSL and ANNS tasks. Blue dots indicate the AS-only baseline. Red dots replace one AS mode dimension with ES mode filtering, using the highest inter-class variance, but without VS shifting, resulting in accuracy loss due to mismatch levels 2 and 3 disturbing current distributions. For ANNS, the green curve shows that shifting VS mitigates current suppression and improves accuracy. For FSL, it combines VS shifting with FAT to reduce energy without accuracy loss. We denote nd as n-dimensional L_∞ filtering per string (e.g., 1d means one-dimensional filtering). Compared to the AS-only baseline, 1d filtering saves 74% and 71% energy on Omniglot and CUB with under 0.5% accuracy loss. On Fashion-MNIST, MNIST, and GIST, it reduces energy by 77%, 83%, and 67%, with under 0.01 recall drop. Compared to TCAM+CIM [15], which filters with TCAM-based L_∞ and computes L_1 norms in CIM, our

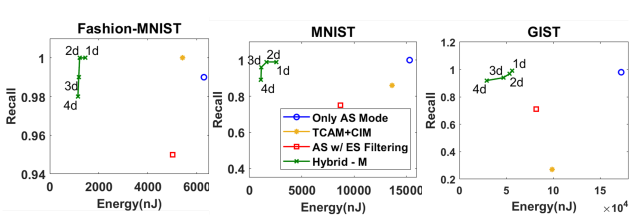

Fig. 10: Energy evaluation of ANNS.

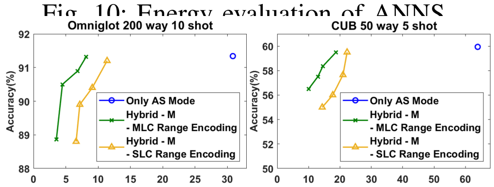

Fig. 11: SLC and MLC range encoding for many-class FSL.

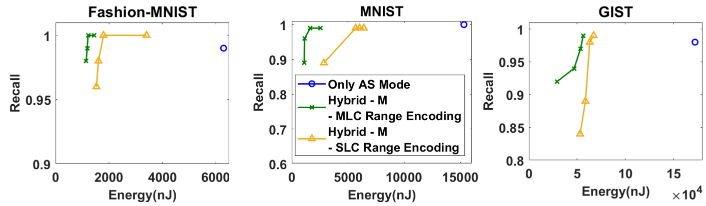

Fig. 12: SLC and MLC range encoding for ANNS.

TABLE VI: Comparison of one-dimensional filtering with VS shifting and the integration of FAT.

Dataset	**Omniglot**		
	Only AS Mode	VS Shifting	FAT
Accuracy	91.34%	90.8%	91.32%
Filtered energy	0%	66%	74%
Dataset	**CUB**		
	Only AS Mode	VS Shifting	FAT
Accuracy	59.94%	58.5%	59.5%
Filtered energy	0%	68%	71%

1d filtering design in Hybrid-M achieves higher accuracy and 10–70% better energy efficiency, demonstrating its advantage in NAND-based MCAM systems.

C. Comparison of Range Encoding between SLC and MLC

Fig. 11 and 12 compare our MLC range encoding with SLC-style methods that use only a subset of MLC states. These approaches underutilize the capacity of MLC, weakening filtering and accuracy. In contrast, our method achieves stronger filtering and higher accuracy, balancing energy efficiency and reliability.

D. Effectiveness of FAT under hybrid matching process

Table VI shows VSS accuracy and energy under different settings. Without training, VS shifting alone causes accuracy degradation. FAT improves both accuracy and energy by jointly learning filtering and VSS. FAT is applicable to trainable scenarios, such as FSL, where the controller is adapted to the hybrid matching process.

V. Conclusion

This work introduces Hybrid-M, a 3D NAND-based in-memory VSS architecture that addresses energy inefficiencies in large-scale VSS. By integrating ES and AS modes, Hybrid-M leverages ES mode filtering to remove redundant comparisons in AS mode, significantly reducing energy consumption while preserving accuracy. We propose three key enhancements–MLC range encoding, VS shifting, and FAT–to optimize hybrid search. Experimental results show that Hybrid-M lowers energy usage by 67–83% compared to MCAM-based VSS using only AS mode in 3D NAND flash, while maintaining accuracy across many-class FSL and ANNS workloads. These findings confirm the scalability and efficiency of Hybrid-M, making it a promising solution for large-scale VSS applications.

Acknowledgment

This work was supported by the National Science and Technology Council of Taiwan under Grants of MOST 111-2218-E-002-018-MBK and NSTC 112-2218-E-002-025-MBK.

REFERENCES

[1] Lu Liu, Tianyi Zhou, Guodong Long, Jing Jiang, and Chengqi Zhang, "Many-Class Few-Shot Learning on Multi-Granularity Class Hierarchy," IEEE Transactions on Knowledge and Data Engineering, vol. 34, no. 5, pp. 2293-2305, 2022.

[2] Jiawei Dang, Yu Zhou, Ruirui Zheng, and Jianjun He, "A Large-Class Few-Shot Learning Method Based on High-Dimensional Features," Applied Sciences, vol. 13, no. 23, pp. 12843, 2023.

[3] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull, "ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms," Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018.

[4] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull, "Reproducibility Protocol for ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Search Algorithms," Information Systems, 2020.

[5] Lewis, Patrick, *et al*., "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Syst. (NeurIPS)*, pp. 9459-9474, 2020.

[6] Ashish Ranjan, Shubham Jain, Jacob R. Stevens, Dipankar Das, Bharat Kaul, and Anand Raghunathan, "X-mann: A crossbar based architecture for memory augmented neural networks," Proceedings of the 56th Annual Design Automation Conference 2019, pp. 1–6, 2019.

[7] Han-Wen Hu, Wei-Chen Wang, Yuan-Hao Chang, Yung-Chun Lee, Bo-Rong Lin, Huai-Mu Wang, Yen-Po Lin, Yu-Ming Huang, Chong-Ying Lee, Tzu-Hsiang Su, and others, "Ice: An intelligent cognition engine with 3D NAND-based in-memory computing for vector similarity search acceleration," 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO), pp. 763–783, 2022.

[8] Harsha Simhadri. Research talk: Approximate nearest neighbor search systems at scale. https://www.youtube.com/watch?v=BnYNdSIKibQ&list=PLD7HFcN7LXReJTWFKYqwMcCc1nZKIXBo9&index=9, 2021.

[9] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. Proceedings of the VLDB Endowment, 2019.

[10] Haitong Li, Wei-Chen Chen, Akash Levy, Ching-Hua Wang, Hongjie Wang, Po-Han Chen, Weier Wan, Win-San Khwa, Harry Chuang, Y.-D. Chih, Meng-Fan Chang, H.-S. Philip Wong, and Priyanka Raina, "SAPIENS: A 64-kb RRAM-Based Non-Volatile Associative Memory for One-Shot Learning and Inference at the Edge," IEEE Transactions on Electron Devices, vol. 68, no. 12, pp. 6637-6643, 2021.

[11] Haitong Li, Wei-Chen Chen, Akash Levy, Ching-Hua Wang, Hongjie Wang, PoHan Chen, Weier Wan, H-S Philip Wong, and Priyanka Raina, "One-shot learning with memory-augmented neural networks using a 64-kbit, 118 GOPS/W RRAMbased non-volatile associative memory," 2021 Symposium on VLSI Technology, pp. 1–2, 2021.

[12] Mohsen Imani, Xunzhao Yin, John Messerly, Saransh Gupta, Michael Niemier, Xiaobo Sharon Hu, and Tajana Rosing, "SearcHD: A memory-centric hyperdimensional computing with stochastic training," IEEE Transactions on Computer Aided Design of Integrated Circuits and Systems, vol. 39, no. 10, pp. 2422–2433, 2019.

[13] Po-Hao Tseng, Feng-Ming Lee, Yung-Chun Li, Yu-Hsuan Lin, Chih-Wei Hu, Chih-Chang Hsieh, Ming-Hsiu Lee, Kuang-Yeu Hsieh, Keh-Chung Wang, and Chih-Yuan Lu, "Multilevel In-Memory-Searching in 3D NAND-Flash Memory," 2022.

[14] Po-Hao Tseng, Tian-Cig Bo, Yu-Hsuan Lin, Yu-Chao Lin, Jhe-Yi Liao, Feng-Ming Lee, Yu-Yu Lin, Ming-Hsiu Lee, Kuang-Yeu Hsieh, Keh-Chung Wang, and Chih Yuan Lu, "SLC and MLC In-Memory-Approximate-Search Solutions in Commercial 48-layer and 96-layer 3D-NAND Flash Memories," the 2022 International Conference on Solid State Devices and Materials, pp. 443-444, 2022.

[15] Laguna, A. F., Yin, X., Reis, D., Niemier, M. & Hu, X. S. Ferroelectric FET based In-Memory Computing for Few-Shot Learning. Proceedings of the ACM Great Lakes Symposium on VLSI, GLSVLSI, pp. 373-378, 2019.

[16] Anat Bremler-Barr, Yotam Harchol, David Hay and Yacov Hel-Or, "Encoding Short Ranges in TCAM Without Expansion: Efficient Algorithm and Applications," IEEE/ACM Transactions on Networking, vol. 26, no. 2, pp. 835-850, 2018.

[17] Chi-Tse Huang, Cheng-Yang Chang, Hsiang-Yun Cheng and An-Yeu Wu, "BORE: Energy-Efficient Banded Vector Similarity Search with Optimized Range Encoding for Memory-Augmented Neural Network," 2024 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 1-6, 2024

[18] Chiang, Hao-Wei, Chi-Tse Huang, Hsiang-Yun Cheng, Po-Hao Tseng, Ming-Hsiu Lee and An-Yeu Wu. "Efficient and Reliable Vector Similarity Search Using Asymmetric Encoding with NAND-Flash for Many-Class Few-Shot Learning." ArXiv abs/2409.07832 (2024).

[19] Reis, D., Laguna, A. F., Niemier, M. & Hu, X. S. Attention-In-Memory for Few-Shot Learning with Configurable Ferroelectric FET Arrays. In Proceedings of the 26th Asia and South Pacific Design Automation Conference, pp. 49–54, 2021.

[20] Ann Franchesca Laguna, Michael Niemier and X. Sharon Hu, "Design of Hardware-Friendly Memory Enhanced Neural Networks," 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE), Florence, Italy, 2019, pp. 1583-1586.

[21] K. J. Eldredge, F. F. Roohparvar, L. De Santis, and T. Vali, "Apparatus and methods of operating memory for exact and inexact searching of feature vectors," U.S. Patent 9,595,334 B2, Mar. 14, 2017.

[22] Brenden M Lake, Ruslan Salakhutdinov, and Joshua B Tenenbaum, "Human-level concept learning through probabilistic program induction," Science, vol. 350, no. 6266, pp. 1332–1338, 2015.

[23] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD Birds-200-2011 Dataset," California Institute of Technology, Technical Report CNS-TR-2011-001, 2011.

[24] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Daan Wierstra, and others, "Matching networks for one shot learning," Advances in Neural Information Processing Systems, vol. 29, 2016.

[25] Boris Oreshkin, Pau Rodríguez López, and Alexandre Lacoste, "Tadam: Task dependent adaptive metric for improved few-shot learning," Advances in Neural Information Processing Systems, vol. 31, 2018.

[26] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," arXiv preprint arXiv:1708.07747, 2017.

[27] L. Deng, "The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]," in IEEE Signal Processing Magazine, vol. 29, no. 6, pp. 141-142, Nov. 2012.

[28] H. Jégou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 1, pp. 117-128, 2011.